

Algorithms & Data Structures

Weeks 1-2: C++ Basics, Complexity, and ADTs

Alexander Bak Thorsen, Viktor Kirkeby Førrisdahl, Bertram Agerskov, Jonas Harwig Mejlholm

September 7, 2026

1 Øvelse

- (A) Write a program that generates random M integers and puts them in an vector. Then it generates another N random integers and counts how many of them are in the array using an iterator:

Vi starter med at lave 2 vektorer med længderne M og N , hvor hvert element er en random integer. Disse kaldes $vecM$ og $vecN$. Initialisering af $vecM$ kan ses i *Code Block 1*. $vecN$ initialiseres på samme måde. Vi vælger at cappe størrelsen på vektorerne til 100 og størrelsen på de tilfældige tal til 50:

```
1  int M = rand() % 101;
2
3  std::vector<int> vecM(M);
4  for (auto i = 0; i < M; i++)
5  {
6      vecM[i] = rand() % 51;
7  }
8
```

Code Block 1: Initialisering af vecM

For at tjekke, hvor mange af elementerne fra $vecN$, der findes i $vecM$ laver vi et for-loop, der itererer over $vecN$, og ved hjælp af $std::find()$ funktionen tjekker om elementet $vecN[i]$ findes i $vecM$. Hvis elementet optræder flere gange i $vecM$, bliver counten kun inkrementeret 1 gang.

Dog kan et element, der forekommer flere gange i $vecN$ godt inkrementere counten flere gange. For-loopet ses i *Code Block 2*.

```
1  int count = 0;
2
3  for (auto i = 0; i < N; i++)
4  {
5      auto it = std::find(vecM.begin(), vecM.end(), vecN[i]);
6
7      if (it != vecM.end())
8      {
9          count++;
10     }
11 }
12
```

Code Block 2: Tællelogikken

(B) Estimate the worst-case complexity of your program and justify with a line-by-line explanation of the number of steps taken in the program:

For at estimere tidskompleksiteten for programmet tæller vi antallet af steps linje for linje. Antallet af steps pr. linje kan ses i *Code Block 3*.

```
1  int main()
2  {
3      std::srand(static_cast<unsigned>(std::time(nullptr))); // 0(1)
4
5      int M = std::rand() % 101; // 1 step
6
7      std::vector<int> vecM(M); // 0(M)
8      for (int i = 0; i < M; ++i) // 1 init + M + 1 checks + M increments
9      {
10         vecM[i] = std::rand() % 51; // M steps: 0(M)
11     }
12
13     int N = std::rand() % 101; // 1 step
14
15     std::vector<int> vecN(N); // 0(N)
16     for (int i = 0; i < N; ++i) // 1 init + N + 1 checks + N increments
17     {
18         vecN[i] = std::rand() % 51; // N steps
19     }
20
21     int count = 0; // 1 step
22
23     for (int i = 0; i < N; ++i) // 1 init + N + 1 checks + N increments
24     {
25         const auto it = std::find( // Worst case: M steps
26             vecM.begin(), // 1 step
27             vecM.end(), // 1 step
28             vecN[i]); // 1 step
29
30         if (it != vecM.end()) // N steps
31         {
32             ++count; // Worst case: N steps
33         }
34     }
35
36     return 0; // 1 step
37 }
```

Code Block 3: Steps pr. linje

Dermed bliver tidskompleksiteten ved det værste tilfælde $O(M) + O(N) + O(NM) = O(N + M + NM)$, hvilket kan forenkles til $O(N*M)$ da de enkelte *1 step*, $O(N)$ og $O(M)$ bliver matematisk insignifikante, når N og M bliver store. Dette kunne forbedres til $O(N+M)$ ved brug af et hash set. Det har vi dog ikke valgt at implementere, da vi endnu ikke har haft om det i undervisningen.

2 Øvelse

(A) What is the time complexity (Big-O) of myMethod? Argue for your answer:

For at finde tidskompleksiteten kan vi begynde at tælle operationer i hele koden og når vi tæller forloops starter vi med den inderste nested forloop og tæller udaf. Optællingen er lavet i kommentarer ud fra hver kodestykke, hvilket kan ses i *Code Block 4*.

```
1  int myMethod(int N)
2  {
3      int x = 0;           // 1
4      int y = 0;           // 1
5      for (int i = 0; i < N; i++)
6      {
7          for (int j = 0; j < N; j++)
8          {
9              for (int k = 0; k < N * sqrt(N); k++)
10             {
11                 x++;       // 1
12             }             // (N * sqrt(N)) * 1
13             }             // ((N * sqrt(N)) * 1) * N
14         }                 // ((N^2.5) * 1) * N
15     for (int i = 0; i < N * N; i++)
16     {
17         y++;               // 1
18     }                     // (N * N) * 1
19     return x + y;         // 1
20 }
21
```

Code Block 4: Steps pr. linje

Derved ender vi med et udtryk, som hedder:

$$T_{body}(N) = 2 + ((N^{2.5}) \cdot 1) \cdot N + (N \cdot N) \cdot 1 + 1 = 3 + N^{3.5} + N^2$$

Da BIG O er defineret som worst case kan vi se på vores udtryk, at tidskompleksiteten for koden ender med at være $O(N^{3.5})$.

3 Øvelse

(A) Analyse af runtime-kompleksitet:

Først analyseres runtime-kompleksiteten ved at undersøge hvor mange gange `++sum` udføres som funktion af n

Fragment (1):

```
sum = 0;
for (i = 0; i < n; ++i) {
    ++sum;
}
```

For-loopet starter ved $i=0$ og forsætter mens $i < n$. Loopet udføres derfor n antal gange. Det vil sige at antal operationer vokser med n : $T(n) = n$. Runtime-kompleksiteten bliver derfor: $O(n)$.

Fragment (2):

```
sum = 0;
for (i = 0; i < n; ++i) {
    for (j = 0; j < n; ++j) {
        ++sum;
    }
}
```

Det ydre loop udføres n antal gange. For hver iteration af det ydre loop, udføres det indre loop n antal gange.

Det vil sige at samlede antal udførelser af `++sum` bliver: $T(n) = n \cdot n = n^2$
Runtime-kompleksiteten bliver: $O(n^2)$

Fragment (3):

```
sum = 0;
for (i = 0; i < n; ++i) {
    for (j = 0; j < n * n; ++j) {
        ++sum;
    }
}
```

Det ydre loop udføres n gange. For hver iteration af det ydre loop, udføres det indre loop n^2 antal gange. Det vil sige at samlede antal udførelser af `++sum` bliver: $T(n) = n \cdot n^2 = n^3$
Runtime-kompleksiteten bliver: $O(n^3)$

Fragment (4):

```
sum = 0;
for (i = 0; i < n; ++i) {
    for (j = 0; j < i; ++j) {
        ++sum;
    }
}
```

Her afhænger antallet af iterationer af det indre loop af i . Så når $i = 0$, kører indre loop 0 gange, når $i = 1$ kører det 1 gang, osv. indtil $i = n - 1$

Det vil sige at samlede antal udførelser af `++sum` bliver: $T(n) = 0 + 1 + 2 + \dots + (n - 1)$

Dette kan skrives i en formel:

$$T(n) = \frac{n(n-1)}{2}$$

Runtime-kompleksiteten bliver: $O(n^2)$

(B) Implementering / måling af fragmenter

Her implementeres de 4 programfragmenter som separate C-funktioner, hvor hver funktion modtager n som parameter og returnerer den beregnede værdi af `sum`

```
1 long long fragment1(int n)
2 {
3     long long sum = 0;
4
5     for (int i = 0; i < n; ++i)
6     {
7         ++sum;
8     }
9     return sum;
10 }
11
12 long long fragment2(int n)
13 {
14     long long sum = 0;
15
16     for (int i = 0; i < n; ++i)
17     {
18         for (int j = 0; j < n; ++j)
19         {
20             ++sum;
21         }
22     }
23     return sum;
24 }
25
26 long long fragment3(int n)
27 {
28     long long sum = 0;
29
30     for (int i = 0; i < n; ++i)
31     {
32         for (int j = 0; j < n * n; ++j)
33         {
34             ++sum;
35         }
36     }
37     return sum;
38 }
39
40 long long fragment4(int n)
41 {
42     long long sum = 0;
43
44     for (int i = 0; i < n; ++i)
45     {
46         for (int j = 0; j < i; ++j)
47         {
48             ++sum;
```

```

49     }
50 }
51 return sum;
52 }

```

Code Block 5: Implementering af fragmenter

Runtime for hver funktion blev målt vha. `clock()` fra `time.h`-biblioteket, ved at registrere et time-stamp før og efter funktionskaldet.

For hvert fragment køres funktionen med adskillige værdier af n , og fordobles mellem hver måling. Herefter kan ændringen i runtime sammenlignes med kompleksiteten fra analysen i del (A).

```

1 clock_t start = clock();
2
3 long long result = fragment1(n);
4
5 clock_t end = clock();
6
7 double elapsed =
8     (double)(end - start) / CLOCKS_PER_SEC;

```

Code Block 6: Eksempel på måling af runtime

(C) Sammenligning med analyse

De målte runtimes sammenlignes med analysen fra opgave (A). Sammenligningen foretages ved at undersøge ændringen i runtime, når inputstørrelsen n fordobles:

$$R = \frac{T(2n)}{T(n)}.$$

For en algoritme med lineær kompleksitet, $O(n)$, forventes en faktor på 2.

For $O(n^2)$ forventes en faktor på ca. 4.

For $O(n^3)$ forventes en faktor på ca. 8.

Fragment	n	Runtime [s]	Faktor
1 $O(n)$	10000000	0.006	–
	20000000	0.012	2.00
	40000000	0.025	2.08
	80000000	0.048	1.92
2 $O(n^2)$	2000	0.003	–
	4000	0.010	3.33
	8000	0.040	4.00
	16000	0.155	3.88
3 $O(n^3)$	100	0.001	–
	200	0.006	6.00
	400	0.039	6.50
	800	0.311	7.97
4 $O(n^2)$	2000	0.002	–
	4000	0.005	2.50
	8000	0.019	3.80
	16000	0.076	4.00

Table 1: Målte runtimes og ændringen i runtime ved fordobling af n .

Overordnet stemmer de målte runtimes overens med den forventede faktor. For fragment 1 ligger faktoren tæt på 2 som forventet for $O(n)$.

For fragment 2 og 4 nærmer faktoren sig 4, som forventet for $O(n^2)$.
 For fragment 3 nærmer faktoren sig 8 når n når de større input-værdier, som forventet for $O(n^3)$.
 De største afvigelser ses ved de korteste målinger.

(D) Compiler-optimering

Programmet blev efterfølgende kompileret med `-O`-optimeringsflag for at undersøge om compiler-optimering ændrer resultaterne

Fragment	n	Uden <code>-O</code> [s]	Med <code>-O</code> [s]
1	80000000	0.048	0.049
2	16000	0.155	0.157
3	800	0.311	0.327
4	16000	0.076	0.090

Table 2: Sammenligning af runtimes med og uden `-O`-optimeringsflag ved de største inputsværdier.

De målte runtimes med og uden compiler-optimering var overordnet ens. Compiler-optimeringen ændrer derfor heller ikke Big-O analysen af fragmenterne.

4 Øvelse

- (A) Write an implementation of the Abstract Data Type MaxHeap described below. The implementation must be done by using the standard class vector from C++ and it does not have to be efficient. If you need to iterate through the vector, you must use an iterator and its methods and operators begin(), end(), ++ and *:

Vi har lavet implementeringen i "Exercise_4" mappen. Vi læser opgaven som om, at vi skal bruge en normal `std::vector` til at løse opgaven med. Derved tænker vi, at vi kan udnytte nogle af alle de funktioner, som allerede findes til netop vektorer i vores implementering.

Designovervejelser

Vi har brugt mange af de funktioner, som kan bruges på vektorer normalt. Eksempelvis `push_back()` i vores insert funktion, da vi har valgt ikke at sortere vores heap, hvilket kan ses i *Code Block 11*:

```
1 void MyMaxHeap::insert(const int x)
2 {
3     heap.push_back(x);
4 }
```

Code Block 7: insert funktion

Når vi sletter vores max element sletter vi kun 1 optræden af den. dvs. at hvis max element opstod flere gange ville de forblive tilbage og kun 1 tilfælde ville blive slettet, hvilket kan ses, da vi har set et break, når max bliver fundet:

```
1 int MyMaxHeap::deleteMax()
2 {
3     if (isEmpty())
4     {
5         throw std::runtime_error("deleteMax() kan ikke kaldes paa en tom heap");
6     }
7
8     int tempMax = findMax();
9     for (auto it = heap.begin(); it != heap.end(); )
10    {
11        if (*it == tempMax)
12        {
13            it = heap.erase(it);
14            break;
15        }
16        else
17        {
18            it++;
19        }
20    }
21
22    return tempMax;
23 }
24
```

Code Block 8: deleteMax funktion

(B) What is the time complexity of the five operations of MaxHeap? You must argue for your answers:

Vi tæller antallet af operationer ligesom i Øvelse 1 og fremskriver worst case, som angiver BIG O notation:

isEmpty():

```
1 bool MyMaxHeap::isEmpty() const
2 {
3     return (heap.begin() == heap.end()); // 1 step
4 }
5
```

Code Block 9: Steps pr. linje for isEmpty funktion

isEmpty() har derfor en tidskompleksitet på $O(1)$.

size():

```
1 int MyMaxHeap::size()
2 {
3     int heapSize = 0; // 1 step
4     if (isEmpty()) // 1 step
5     {
6         return heapSize; // 1 step
7     }
8     else
9     {
10        for (auto it = heap.begin(); it != heap.end(); )
11        {
12            heapSize++; // 1 step
13            it++; // 1 step
14        } // 1 + (N * 2)
15        return heapSize; // 1 step
16    }
17 }
18
```

Code Block 10: Steps pr. linje for size funktion

size() har derfor en tidskompleksitet på $O(N)$.

insert():

```
1 void MyMaxHeap::insert(const int x)
2 {
3     heap.push_back(x); // 1 step
4 }
5
```

Code Block 11: Steps pr. linje for insert funktion

insert() har derfor en tidskompleksitet på $O(1)$, men kan dog nogle gange godt have $O(N)$, da hvis den plads, som er allokeret på heapen til vektoren ikke er stor nok til at få et element mere skal hele vektoren rykkes til et andet sted og det tager $O(N)$ alt efter hvor mange elementer N , som allerede var i vektoren.

findMax():

```
1 const int MyMaxHeap::findMax() const
2 {
3     if (isEmpty()) // 1 step
4     {
5         throw std::runtime_error("findMax()");
6     }
7 }
```

```

6     kan ikke kaldes paa en tom heap");           // 1 step
7 }
8 int max = heap[0];                             // 1 step
9 for (auto it = heap.begin(); it != heap.end();) // n steps
10 {
11     if (*it > max)                               // n steps
12     {
13         max = *it;                             // n steps
14     }
15     it++;                                       // n steps
16 }
17 return max;                                   // 1 step
18 }
19

```

Code Block 12: Steps pr. linje for *findMax* funktion

Funktionen er $O(n)$, fordi den skal gennemløbe hele listen og sammenligne hvert element for at finde det største.

deleteMax():

```

1 int MyMaxHeap::deleteMax()
2 {
3     if (isEmpty())                             // 1 step
4     {
5         throw std::runtime_error("deleteMax() kan ikke
6         kaldes paa en tom heap");             // 1 step
7     }
8
9     int tempMax = findMax();                   // n steps
10    for (auto it = heap.begin(); it != heap.end();) // n steps
11    {
12        if (*it == tempMax)                   // n steps
13        {
14            it = heap.erase(it);              // n steps
15            break;                            // 1 step
16        }
17        else
18        {
19            it++;                             // n steps
20        }
21    }
22
23    return tempMax;                           // 1 step
24 }
25

```

Code Block 13: Steps pr. linje for *deleteMax* funktion

Funktionen er $O(n)$ tid, da *findMax()* og *erase()* begge tager n steps. Alle andre operationer tager konstant tid.

5 Øvelse

(A) Create and implement a C++ class called *Library*:

Implementering af *Library*-klassen kan ses i *Code Block 14*.

```
1  #include "library.h"
2
3  Library::Library() {}
4
5  void Library::addBook(int id, int category)
6  {
7      if (valid_categories_ > 0)
8      {
9          if (((static_cast<float>(books_.size()) / valid_categories_) * 2) < categories_[category] + 1)
10             {
11                 std::cout << "\n"
12                     << "Warning! Category " << category << " exceeds twice the average number of books per
13                     valid category.\n";
14             }
15         }
16
17         if (categories_[category] == 0)
18         {
19             valid_categories_++;
20         }
21
22         categories_[category]++;
23
24         books_.push_back(Book(id, category));
25     }
26
27 void Library::displayAllBooks()
28 {
29     for (const auto &i : books_)
30     {
31         std::cout << i;
32     }
33 }
34
35 void Library::displayAvailableBooks()
36 {
37     for (const auto &i : books_)
38     {
39         if (!i.getStatus())
40         {
41             std::cout << i;
42         }
43     }
44 }
45
46 void Library::displayBorrowedBooks()
47 {
48     for (const auto &i : books_)
49     {
50         if (i.getStatus())
51         {
52             std::cout << i;
53         }
54     }
55 }
56
57 Borrowed_Status Library::borrowBook(int id)
58 {
59     if (books_[id - 1].getStatus() == false)
60     {
```

```

60     books_[id - 1].updateStatus(true);
61     return Borrowed_Succes;
62 }
63 else
64 {
65     return Borrowed_Not_succes;
66 }
67 }
68
69 Return_Status Library::returnBook(int id)
70 {
71     if (books_[id - 1].getStatus() == true)
72     {
73         books_[id - 1].updateStatus(false);
74         return Return_Succes;
75     }
76     else
77     {
78         return Return_Not_succes;
79     }
80 }
81
82 int Library::countBooksInCategory(int category)
83 {
84     return categories_[category];
85 }

```

Code Block 14: Implementering af Library-klassen

(B) Justify the choice of the data structures you chose to maintain the books in the library:

Vi har taget vores beslutninger ud fra *Figure 1*, som kan ses i bunden af dokumentet.

Vi har valgt at bruge 2 data strukturer til opgaven, en vector til at holde styr på bøgerens id og deres kategorier og et array til at holde styr på antallet af brugte kategorier og antallet af bøger i hver kategori.

Valget af vektoren til at holde styr på bøgerne blev taget, da vi ikke på forhånd kender antallet af bøger der tilføjes. Derudover da vi kun skal kunne tilføje, låne og retunere bøger, og ikke slette bøger, var en vektor optimal da den dynamisk kan allokere plads til nye bøger. og man kan søge via et indeks

Vi valgte et array, fordi antallet af kategorier er kendt på forhånd. Derfor er der ikke behov for dynamisk resizing, og kategorierne kan tilgås direkte med et indeks.

(C) Provide the complexity of each operation in the library, and justify:

Vi lader her n betegne antallet af bøger i biblioteket. Funktion *addBook()* har en amortiseret tidskompleksitet på $O(1)$, da beregningerne samt adgang til og opdatering af *categories_* tager en konstant tid, og *push_back()* på en vektor i de fleste tilfælde er konstant, og i nogle få tilfælde kan være $O(n)$ hvis vektoren skal udvide sin kapacitet.

Funktionerne *displayAllBooks()*, *displayAvailableBooks()* og *displayBorrowedBooks()* alle har tidskompleksiteten $O(n)$, da de hver især gennemløber hele *books_* vektoren en gang.

Funktionerne *borrowBook()* og *returnBook()* har de tidskompleksiteten $O(1)$ da de tilgås direkte efter deres id med *books_[id-1]* Dette forudsætter at bogens ID svarer til placeringen i vektoren.

I Funktionen *countBooksInCategory()* er tidskompleksiteten $O(1)$, da antallet af bøger i en kategori gemmes direkte i arrayet *categories_* og derfor kan findes ved indeks.

<i>addBook()</i>	$O(1)$
<i>displayAllBooks()</i>	$O(n)$
<i>displayAvailableBooks()</i>	$O(n)$
<i>displayBorrowedBooks()</i>	$O(n)$
<i>borrowBook()</i>	$O(1)$
<i>returnBook()</i>	$O(1)$
<i>countBooksInCategory()</i>	$O(1)$

Table 3: Oversigt over funktioner i library

Operation	<code>std::vector</code>	<code>std::deque</code>	<code>std::list</code>
Insert back	<code>push_back</code> <code>emplace_back</code>	<code>push_back</code> <code>emplace_back</code>	<code>push_back</code> <code>emplace_back</code>
Insert front	<i>Not implemented</i>	<code>push_front</code> <code>emplace_front</code>	<code>push_front</code> <code>emplace_front</code>
Access ends	<code>back()</code> <code>front()</code>	<code>back()</code> <code>front</code>	<code>back()</code> <code>front</code>
Insert middle	<i>Expensive</i>	<i>Expensive</i>	<i>Efficient</i>
Delete back	<code>pop_back()</code>	<code>pop_back()</code>	<code>pop_back()</code>
Delete front	<i>Not implemented</i>	<code>pop_front</code>	<code>pop_front</code>
Access by index	<code>at(i)</code> <code>[i]</code>	<code>at(i)</code> <code>[i]</code>	<i>Not implemented</i>
Iteration	<i>Efficient</i>	<i>Efficient</i>	<i>Efficient (Bidirectional)</i>
Memory control	<code>capacity()</code> <code>reserve()</code> <code>shrink_to_fit()</code>	<code>shrink_to_fit()</code>	<i>Not necessary</i>
Remove data	<code>clear()</code>	<code>clear()</code>	<code>clear()</code>
Size	<code>size()</code> <code>empty()</code>	<code>size()</code> <code>empty()</code>	<code>size()</code> <code>empty()</code>

Figure 1: Billede med STL containere